Performance Evaluation of Serverless Computing for Image Processing: AWS Lambda vs. Google Cloud Functions

Authors: Matthew Anthony TJOA (57273836), Yaochao YAN (57854065), Zimeng WAN (57222904)

Abstract

The serverless computing offers automatic scaling and a pay-per-use pricing model. It eliminates the need for server maintenance. For this, serverless computing has transformed cloud infrastructure. In our project, we evaluate the performance of two Function-as-a-Service (FaaS) platforms: AWS Lambda and Google Cloud Functions (GCP). To better compare the performance of these two platforms, we run the computationally demanding and highly parallelizable image processing program on each of these two platforms. The image processing program includes comprising resizing, grayscale conversion, and Gaussian blur filtering. In this project, we deploy identical Python-based containerized artifacts on these two platforms. For performance measurement, we take metrics including cold-start latency, warm execution consistency, and stability under high concurrency. Moveover, we conduct comparative cost analysis for processing images with different sizes. GCP has been shown to have an edge over AWS in terms of cold-start latency (2.65s compared to 4.39s) and warm latency for small images (0.57s compared to 0.64s). AWS has demonstrated a significantly lower latency on average under high concurrency at 500 concurrent calls (4.03s compared to 5.11s). On the other hand, the percentage of errors starts to diverge considerably as the number of concurrent calls increases; AWS produces a much higher error rate of 14.6% with 500 concurrent calls, compared to GCP's 43.1%. The errors from GCP were largely due to processing large images (81.2%).

1. Introduction

Due to Serverless computing's event-driven paradigm, it has become more and more popular. It allows developers to focus on code rather than infrastructure. Among all providers, AWS Lambda is famous among enterprises for its reliability and its ability to handle complex workloads. Also, Google Cloud Functions is also famous for its developer experience and rapid cold starts.

The project will compare these two platforms' performance. We selected the image processing program as our benchmark workload. The image processing tasks, such as image resizing,grayscale conversion, and Gaussian blur filtering, are CPU-heavy, bursty, and highly parallelizable. Also, the image processing tasks are also frequently used in real-world applications. Therefore, it is ideal for testing FaaS environments.

2. Methodology and Implementation

To make sure our comparison is fair and reproducible, we use Docker in this project to ensure that the environment is consistent across two cloud platforms.

2.1 Workload Design and Dataset

The program is developed using Python and Python Pillow Library. In our image processing program, an input image will go through three sequential transformations, including resizing, grayscale conversion, and applying a Gaussian blur filter.

In our dataset, we find images with different sizes to evaluate on how input image's size affects execution time and memory limits. There are three types of images:

- **Small Tier (< 500 KB):** These images are responsible for testing baseline invocation and network overhead.
- **Medium Tier (1 MB - 3 MB):** These images are responsible for simulating standard social media image uploads.
- **Large Tier (5 MB - 10 MB):** These images are responsible for testing CPU-intensive processing.

2.2 AWS Lambda Deployment

The SAM model uses a source-based (Zip) deployment workflow for the Amazon Web Services (AWS) serverless application [1]. The SAM deployment tool will read the configuration from the template.yaml file, package the Python source code together with the associated dependencies, and deploy the function using AWS CloudFormation. The Dockerfile at the top level of the repository is just for local testing via docker run and is not included in the cloud deployment process. This function will be deployed in the us-east-1 region.

2.2.1 SAM Deployment Configuration

The deployment process is specified in aws/template.yaml, which is an AWS SAM (Serverless Application Model) template that defines the Lambda function with Runtime: python3.11, Handler: lambda_function.lambda_handler, and CodeUri: . (the source directory). Requirements for dependencies are specified in requirements.txt and packed into the deployment package by SAM:

- **Package Source Code:** SAM builds the handler written in Python (lambda_function.py) and requirements in requirements.txt file into a deployment package compatible with the x86_64 architecture.
- **Deploy using CloudFormation:** Using the sam build and sam deploy command in SAM CLI will allow compiling the package and deploying to AWS using CloudFormation by creating or modifying the resources associated with the stack.
- **Function Configuration:** As per the template configuration, the runtime will be python3.11, which ensures the same execution environment for every run. Handler entry point: lambda_function.lambda_handler

2.2.2 Function Provisioning and Triggering

The Lambda function was created using the **Zip (source-based)** deployment type. The following hardware and runtime configurations were applied to standardize the benchmark:

The SAM template provisions the Lambda function with the following hardware and runtime configurations to standardize the benchmark:

- **Memory Allocation**: It is fixed at **512 MB**. This memory size can provide enough CPU power for Gaussian blur operations especially on our Large tier images. At the same time, it is also cost-effective.
- **Architecture**: We set it to **x86_64** so that it can align with our local Docker build and the standard compute instances.
- **Timeout Settings**: The timeout was set to 60 seconds as defined in the template.yaml file. It is important to take note that the scripts/benchmark.py script has a different HTTP client timeout of 120 seconds.

2.2.3 Interface Integration via API Gateway

To expose the function for automated benchmarking, we use an **AWS API Gateway (REST API)**.

- **Endpoint Configuration**: We create A POST method to receive JSON payloads from the benchmark.py script.
- **Security and Access**: The API was deployed to a "Prod" stage. This provides a public HTTPS endpoint.
- **Data Flow**: The Lambda function was granted IAM permissions to read from the dedicated **S3 Bucket**. The 9-image dataset is stored in this **S3 Bucket**. The function retrieves the image via the key provided in the API request. Then the function processes it, and returns the execution duration metadata to the client.

2.3 Google Cloud Functions Deployment

We use **Cloud Functions (2nd Gen)** to deploy our process on Google Cloud Platform (GCP) [2]. This is built on top of Cloud Run and Eventarc. To make sure that the performance comparison with AWS Lambda is fair, we use the same recourse allocation wherever possible.

2.3.1 Environment and Runtime

We execute the program in the us-central1 region. The Python we use is the latest 3.11 version. The GCP deployment focused on a source-based workflow:

- **Dependency Management**: We defined the library information in requirements.txt. The Google Cloud's build service automatically reads the requirements.txt during the deployment process.
- **Containerization**: GCP automatically containerizes the code and stores the resulting image in the **Artifact Registry**. This makes sure that the execution environment is immutable and reproducible.

2.3.2 Resource Configuration

To make sure the computational power is same as the AWS Lambda instance, the resource parameters are configured as below:

- **Memory Allocation**: It is set to **512 MB**. In GCP's 2nd Gen functions, this memory tier is automatically coupled with approximately **0.333 vCPU**. This provides the necessary processing power for the Gaussian blur and filtering operations.
- **Instance Scaling**: The function was configured to allow minimum instances of zero. This makes sure that it can accurately measure **Cold Start** latencies during the idle-period testing phase.

2.3.3 Triggering and Public Access

The function, named process_image, was exposed via a **HTTP Trigger**.

- **Invocation Mechanism**: The benchmark.py script invokes the function via a secure HTTPS URL.
- **Authentication**: During performance evaluation, "Allow unauthenticated invocations" is enabled. It facilitates direct testing from the benchmarking suite. At the same time, it ensures the POST payload format is identical to the AWS.
- **Response Handling**: In this part, the program returns a JSON response containing the duration_seconds and image metadata, which is then parsed and logged by our local monitoring tools.

2.4 Benchmarking Strategy

For our evaluation framework in benchmark.py, we use a multi-tier testing paradigm. This can simulate real-world bursty and concurrent workloads.

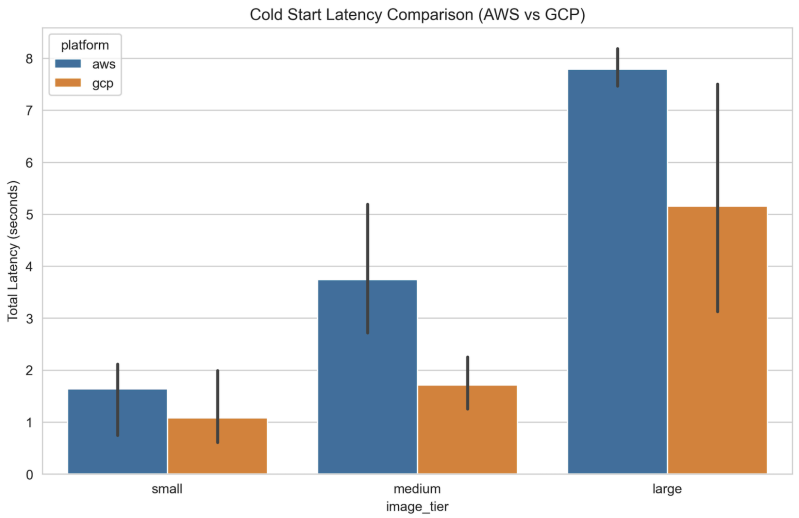
- **Cold Start Test**: This part measures the latency of the first invocation after an idle period of 30+ minutes. It captures the container initialization overhead.
- **Warm Start Test**: This part executes 100 sequential requests per image tier. It is used to calculate average execution consistency.
- **Concurrency Test**: This part utilizes asyncio to fire 50, 100, and 500 simultaneous requests. This measures the platform's ability to scale elastically. This part also identifies the "long-tail" latency (p95, p99) under severe load.

3. Performance Evaluation

3.1 Cold Start Latency Analysis

Cold start latency is defined as the amount of time elapsed between the invocation of the application when it is in the idle state until the first response from the invocation is returned, including all the startup overhead involved in initializing the container and getting it ready [3].

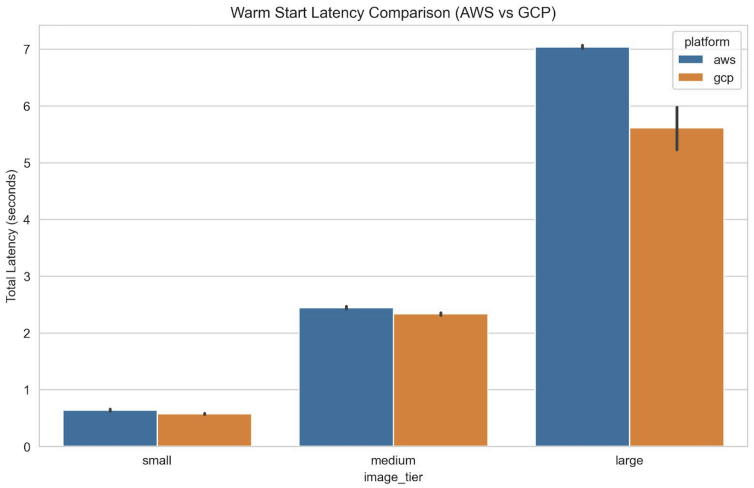
For each image tier, the GCP service showed significantly less average cold start time (2.65 s) than AWS (4.39 s), indicating a 40% advantage for GCP. This result is in line with the Cloud Function 2nd Gen of GCP being implemented on Cloud Run, which is built for fast container boot-up. In contrast, the AWS Lambda function deployed with AWS SAM faces a higher initialisation cost, presumably owing to the boot-up process of the Python 3.11 interpreter and importing libraries from the deployment package.



3.2 Warm Execution Performance

Warm execution times were determined through the issuance of 100 consecutive requests per level of images after initialization of the function. It is evident from the findings that although both platforms increase their execution time depending on the complexity of images, the scaling process for each platform varies at each level of images.

GCP wins in case of small images with a time of 0.57s compared to AWS with 0.64s. In the case of medium images, GCP once again proves its dominance with a time of 2.33s compared to AWS with 2.44s. When it comes to big images, GCP once again proves superior with a time of 5.61s compared to AWS with 7.04s, which makes it about 20% faster.

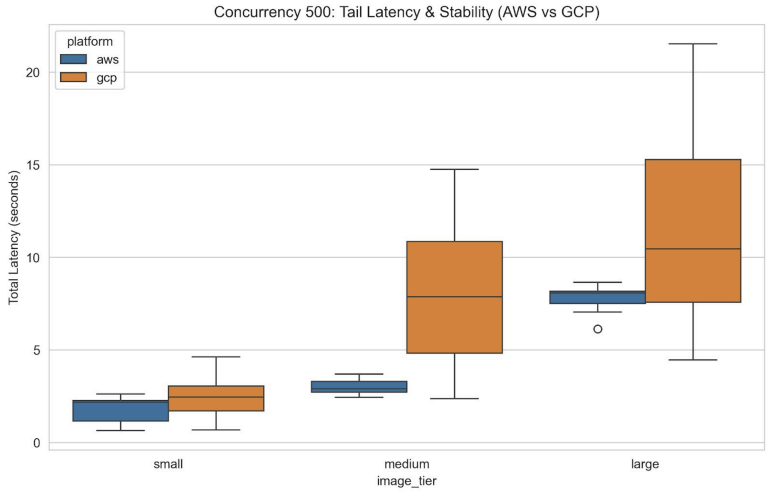


3.3 Concurrency and Stress Testing

The concurrency stress test executed 50, 100, and 500 parallel queries by leveraging the asyncio library in Python. Under low concurrency (50 queries), both cloud providers successfully processed all queries without any errors. In terms of latency under such concurrency, AWS performed slightly better with 4.13 seconds compared to GCP at 4.97 seconds. Under medium concurrency (100 queries), AWS processed all queries without errors, but GCP encountered errors with an error rate of 14.0%. Under high concurrency (500

queries), AWS experienced a higher error rate of 14.6%, whereas GCP had a significantly higher error rate of 43.1%.

GCP's shortcomings on concurrency_500 were largely biased based on image tier; GCP experienced no errors when processing small images but registered an error rate of 48.2% for medium-sized images and 81.2% for large images. The results clearly indicate that GCP fails to allocate enough CPU power for the CPU-heavy large image process when a significant number of instances have to be launched simultaneously. AWS had similar error rates across the different tiers. See Table 1 below.



Concurrency Level	AWS Error Rate	GCP Error Rate
50	0%	0%
100	0%	14.0%
500	14.6%	43.1%

Table 1. Error Rates Under Concurrent Load

4. Cost Analysis

4.1 Pricing Models Comparison

AWS Lambda supports two processor architectures: traditional **x86_64** (Intel/AMD) and **arm64** (ARM) based on the AWS Graviton processor. The billing methods for these two architectures differ, primarily in the price per GB-second. The x86_64 architecture uses standard pricing, while the arm64 (Graviton) architecture typically costs about **20%** less than x86 and delivers the same or better performance in many CPU-intensive workloads (such as graphics processing), resulting in a higher price-performance ratio (up to **34%** improvement). To maintain consistency with Google Cloud Run functions' billing model, this project will uniformly use the **x86_64** architecture for deployment and experimentation. The charging rules are shown in the following figure.

AWS Lambda Pricing

Region:

US East (N. Virginia)

Architecture	Duration	Requests
x86 Price		
First 6 Billion GB-seconds / month	$0.0000166667 for every GB-second	$0.20 per 1M requests
Next 9 Billion GB-seconds / month	$0.000015 for every GB-second	$0.20 per 1M requests
Over 15 Billion GB-seconds / month	$0.0000133334 for every GB-second	$0.20 per 1M requests
Arm Price		
First 7.5 Billion GB-seconds / month	$0.0000133334 for every GB-second	$0.20 per 1M requests
Next 11.25 Billion GB-seconds / month	$0.0000120001 for every GB-second	$0.20 per 1M requests
Over 18.75 Billion GB-seconds / month	$0.0000106667 for every GB-second	$0.20 per 1M requests

Cloud Functions (2nd Gen) support two main billing models: **Request-based billing** (the default model, billed based on vCPU and memory usage during request processing) and **Instance-based billing** (billed over the entire instance lifecycle, incurring charges even when idle). Additionally, Cloud Run provides **Jobs** (for batching one-off tasks) and **Worker Pools** (for continuously running workers in the background), but neither of these is suitable for the **HTTP-triggered** image processing scenario of this project. This project uses the default **Request-based billing** to achieve true pay-as-you-go billing and to provide a fair comparison with AWS Lambda's On-Demand billing model. The charging rules are shown in the following figure.

Services (billed by request)

A service that uses request-based billing for the lifetime of the billing instance

Free tier (based on US-central1 active price):

- CPU - Free for the first **180,000 vCPU seconds** per month
- RAM - Free for the first **360,000 GiB seconds** per month
- Requests - **2 million requests per month for free**

Iowa (us-central1) — Show discount options

Resources	Type	Default (USD)
CPU (per vCPU second)	Active time	US$0.000024
	Idle time (minimum number of instances.)[1)]	US$0.0000025
Memory (per GiB second)	Active time	US$0.0000025
	Idle time (minimum number of instances.)[1)]	US$0.0000025
Number of requests (per 1,000,000)	Not available	US$0.40

Comparison between two platforms:

Billing Item	**AWS Lambda (x86_64, On-Demand)**	**Google Cloud Run (Request-based)**

Request Fee	$0.20 per 1,000,000 requests	$0.40 per 1,000,000 requests
Compute Unit Price	$0.0000166667 per GB-second	$0.000024 per vCPU-second $0.0000025 per GB-second
Free Tier (monthly)	1 million requests + 400,000 GB-seconds	2 million requests + 180,000 vCPU-seconds + 360,000 GB-seconds

4.2 Projected Cost for Executions (1,000,000 Images)

To estimate the cost of processing 1,000,000 images, we use the average processing seconds collected from our warm-start and concurrency experiments, along with the actual deployed resource configurations.

Assumptions:

- AWS Lambda: x86_64 architecture, On-Demand mode, allocated memory = **512MB=0.5GB**
- Google Cloud functions: Request-based billing, allocated **0.333 vCPU** and **0.5 GB** memory

Main Cost Calculation (based on Warm Start averages):

Image Tier	**Platform**	**Request Cost (USD)**	**Avg. Processing Time (s)**	**Compute Cost (USD)**	**Total Cost (USD)**
Small	AWS	$0.20	0.1426	$1.19	$1.39
Medium	AWS	$0.20	1.9049	$15.87	$16.07
Large	AWS	$0.20	6.3869	$53.22	$53.42
Small	GCP	$0.40	0.0884	$0.82	$1.22
Medium	GCP	$0.40	1.7787	$16.44	$16.84
Large	GCP	$0.40	4.7480	$43.89	$44.29

5. Conclusion and Future Work

Performance evaluations were carried out for AWS Lambda and Google Cloud Functions (2nd Generation) based on a CPU-heavy image processing task containing three functions: image resize by 50%, gray scaling, and Gaussian filter. The same resource allocation was made for both clouds, which is 512 MB memory per ~0.333 vCPU, with AWS using SAM deployment based on source code running in us-east-1 while GCP used source-based Cloud Functions in us-central1.

Our major findings include the following. We found that GCP showed higher efficiency in terms of the cold start time (2.65s versus 4.39s), as well as reduced warm latency execution for all image sizes under sequential request testing. In contrast, AWS performed much better in case of a high level of concurrency; namely, when the number of concurrent requests was 500, GCP had the error rate of 43.1% against 14.6% from AWS, with most errors occurring with the heavy-weight images requiring substantial CPU (81.2% error rate). Moreover, under 100 or more concurrent requests, AWS showed superiority to GCP in terms of the average response time. The costs of running services on both platforms were relatively similar.

To conclude, GCP Cloud Functions is preferable to AWS Lambda for tasks that have higher requirements for latency and lower/middle-levels of concurrency like real-time image processing pipelines, whereas AWS Lambda is more suitable for large-scale, batch tasks, and higher concurrency. Future research might include testing both options using ARM architecture (Graviton/AWS T2A), using provisioned concurrency in AWS to minimize cold start effects, a greater number of images, and different types of image processing algorithms.

References

[1] Amazon Web Services, "AWS Lambda Developer Guide," *AWS Documentation*, 2026. [Online]. Available: https://docs.aws.amazon.com/lambda/

[2] Google Cloud, "Cloud Functions Documentation," *Google Cloud*, 2026. [Online]. Available: https://cloud.google.com/functions/docs

[3] L. Wang, M. Li, Y. Zhang, T. Ristenpart, and M. Swift, "Peeking Behind the Curtains of Serverless Platforms" in *USENIX Annual Technical Conference (ATC)*, 2018, pp. 133–146.

Artifact Appendix

This section shows the steps to reproduce our local testing environment and deploy the evaluation framework.

1. Metadata

- Project Repository: https://github.com/zmw1012/CS4296-Serverless-Image-Processing.git
- Programming Language: Python 3.9+
- Core Dependencies: Pillow (Image Processing), aiohttp, requests (Benchmarking)
- Automation Framework: Docker

2. Description

Our artifact has a complete set of programs to benchmark image processing performance on AWS Lambda and Google Cloud Functions. It has the core processing logic, a dataset of 9 images, and a comprehensive benchmarking script (benchmark.py). The benchmark.py is capable of measuring cold/warm starts and high concurrency.

2.1 Hardware/Software Dependencies

- Hardware: The containerized environment ensures compatibility with x86_64 cloud architectures.
- Docker: Docker Desktop is required to build and run the portable workflow.
- Cloud Access: Access to AWS (API Gateway & S3) and GCP (Cloud Functions) endpoints is required for remote benchmarking.

3. Installation and Environment Setup

To ensure reproducibility and avoid manual dependency configuration, we provide a portable Docker workflow:

Step 1: Clone the Repository.

git clone https://github.com/zmw1012/CS4296-Serverless-Image-Processing.git

cd CS4296-Serverless-Image-Processing

Step 2: Build the Container Environment.

docker build --platform linux/amd64 -t cloud-project .

4. Experiment Workflow

The evaluation is divided into two phases: local logic validation and remote cloud benchmarking.

4.1 Local Validation (Artifact Integrity)

Run the following command to verify that the image processing logic works correctly on the 9-image dataset:

docker run --rm cloud-project

4.2 Remote Cloud Benchmarking

The scripts/benchmark.py script is used to collect the data presented in this report. Use the following commands to replicate our results (requires valid AWS/GCP endpoints):

 # Run all tests (Cold, Warm, Concurrency)

python benchmark.py --mode all --aws-url <URL> --aws-bucket <BUCKET> --gcp-url <URL>

5. Expected Results

At the end of the evaluation, a results.csv file will be generated in the root directory.

- Console Output: A summary table will be printed, displaying the average, min, p50, and p95 latencies for each tier.
- Data Consistency: The latencies should reflect the tiered nature of the dataset, where "Large" tier images consistently show higher processing times compared to "Small" tier images.